

CLASSICAL PLANNING

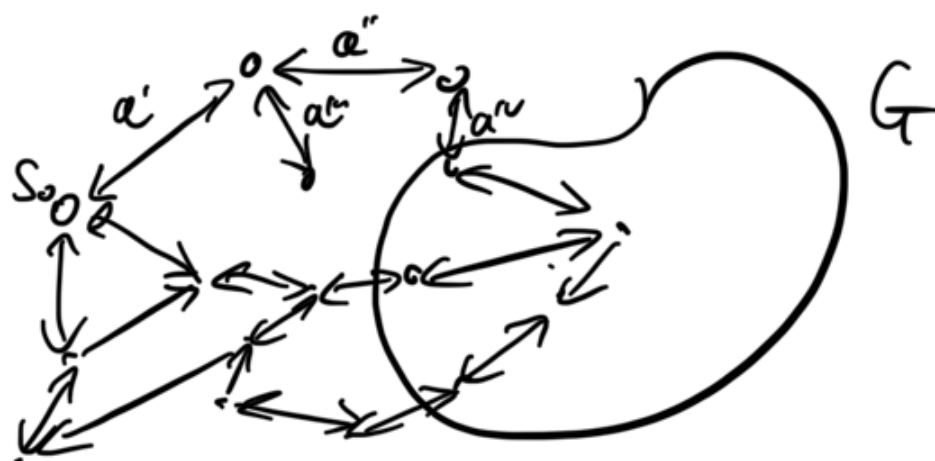
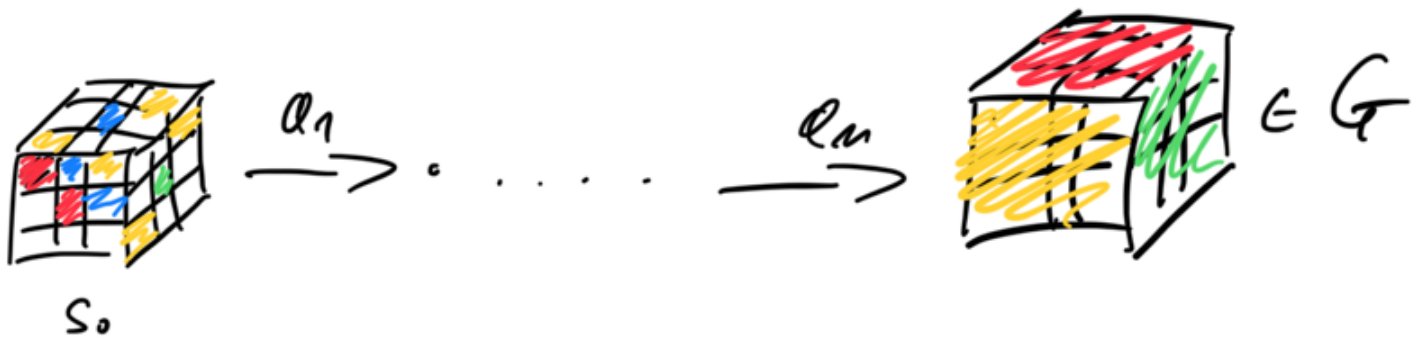
$$\delta(s, a) = \textcircled{s'}$$

Search Algorithms (Best-First, A*)

$$T = \langle S, A, \delta, C, G, s_0 \rangle$$

$$C: S \times A \rightarrow \mathbb{R}$$

$$s_0 \xrightarrow{a_1} s_1 \rightarrow \dots \xrightarrow{a_n} s_n \in G$$



Cost from m_0 to m

$$f(m) = \overbrace{g(m)}^{\text{Cost from } m_0 \text{ to } m} + \underbrace{h(m)}_{\text{estimated cost of reaching some } m \in G \text{ from } m}$$

$h(m)$ ADMISSIBLE $\Rightarrow$ A* returns optimal solution

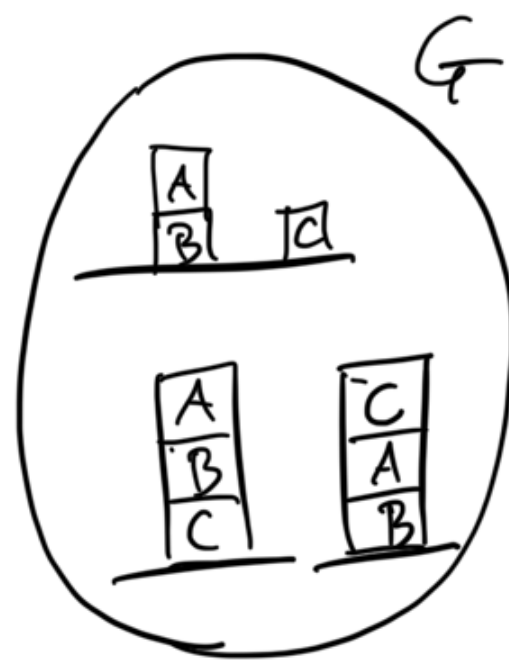
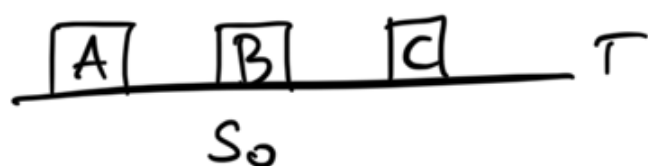
$\uparrow$
Heuristic function

- States do not have a structure
- PLANNING: Factored Representation

FACTORED REPRESENTATION

- World state represented by a set of FLUENTS

- BLOCKSWORLD EXAMPLE



$$\text{FLUENTS} = \{ \text{on-table-a}, \text{on-table-b}, \text{on-table-c}, \\ \text{on-a-b}, \text{on-b-c}, \dots, \text{on-c-a}, \dots \}$$

$$S_0 = \{ \text{on-table-a}, \text{on-table-b}, \text{on-table-c} \}$$

- CWA
- FULLY OBSERVABLE
- DETERMINISTIC ENVIRONMENT

$$\text{FLUENTS} = \{ \text{onTable}(x), \text{on}(x, y), \dots \} \quad \begin{matrix} \boxed{\text{OBSERV}} \\ x, y \in \{A, B, C\} \end{matrix}$$

$$S_0 = \{ \text{onTable}(A), \text{onTable}(B), \text{onTable}(C) \}$$

$$\boxed{\text{onTable}(A)}$$

$$S \subset 2^{\text{FACTS}}$$

onTable (B)
onTable (C)

S₀

$A = \{ \text{move}(x, y), \dots \}$

move (A, B)

GROUND ACTION

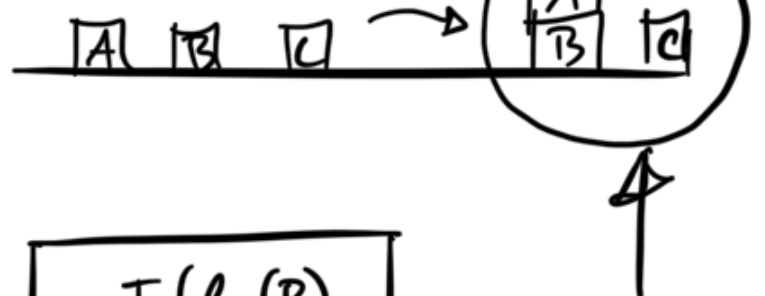
onTable (A)
onTable (B)
onTable (C)

S₀

move (A, B)

onTable (B)
onTable (C)
on (A, B)

onTable (X)
on (x, y)
clear (x)



GROUND FORMS
ATOMS

$\begin{matrix} 3 & 3 \\ \vdots & \vdots \\ \text{on-a-b} & \dots \\ \text{on-b-c} & \\ \vdots & \\ \text{on-c-c} & \end{matrix} \left. \vphantom{\begin{matrix} 3 & 3 \\ \vdots & \vdots \\ \text{on-a-b} & \dots \\ \text{on-b-c} & \\ \vdots & \\ \text{on-c-c} & \end{matrix}} \right\} g$

FACTS

$S = 2^{\text{FACTS}}$

on(x)

OBJ = {A, B, C}

$\begin{matrix} \text{move-a-b} \\ \text{move-a-c} \end{matrix} \left(\begin{matrix} |\text{OBJ}|^M \\ a \end{matrix} \right)$

move (x)

$x \in \{A, B, C\}$

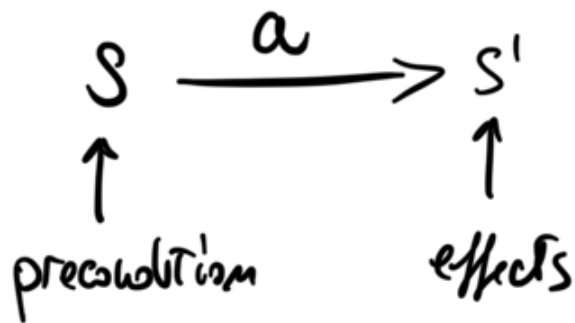
move : c



ACTION SCHEMAS

- Action:

- issue : move
- parameters : x, y
- precondition : necessary conditions for action to be executable in current state
- effects : specify what changes AFTER action execution

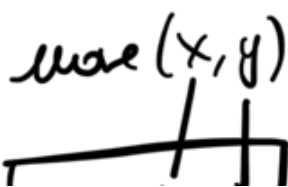


• preconditions:



$\text{move}(x, y):$

• precondition: $\text{Clear}(x) \wedge \text{Clear}(y)$



$(S) \xrightarrow{\text{move}(A,B)}$

$\text{Clear}(A)$
 $\text{Clear}(B)$
 $\text{OnTable}(A)$
 $\text{OnTable}(B)$
 $\text{OnTable}(C)$

$[S_0]$

$\xrightarrow{\text{move}(A,B)}$

$\overline{\text{Clear}(A) \wedge \text{Clear}(B)}$

$(S_0) \models ? \overline{\text{Clear}(A) \wedge \text{Clear}(B)}$

EFFECT

- DETERMINISTIC environment •

$\text{move}(x, y):$

$\vdots$

effect: $\neg \text{OnTable}(x) \wedge \text{On}(x, y) \wedge \neg \text{Clear}(y)$

$\text{Clear}(A)$
 $\text{Clear}(B)$
 $\text{OnTable}(A)$
 $\text{OnTable}(B)$
 $\text{OnTable}(C)$

S

$\xrightarrow{\text{move}(A,B)}$

$S' = (S - \text{DEL}) \cup \text{ADD}$

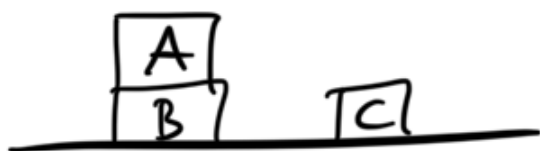
$S - \text{DEL} = \begin{cases} \text{Clear}(A) \cdot \cancel{\text{OnTable}(A)} \\ \text{OnTable}(B) \cdot \cancel{\text{Clear}(B)} \\ \text{OnTable}(C) \end{cases}$
 $\text{On}(A, B)$
 S'

$\neg \text{OnTable}(A), \text{On}(A, B), \neg \text{Clear}(B)$

$\text{ADD} = \{ \text{On}(A, B) \}$

$$DEL = \{ OnTable(A), Clear(B) \}$$

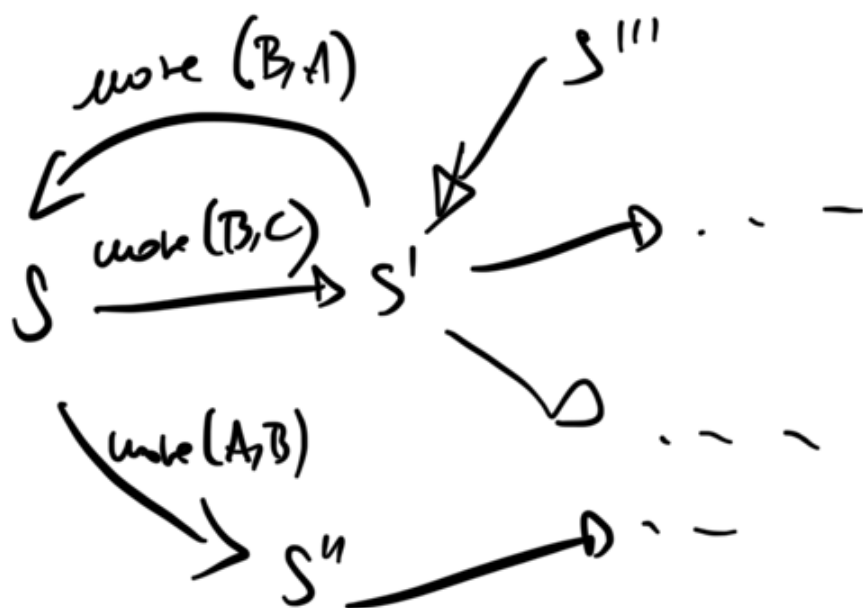
$$S' = (S \setminus DEL) \cup ADD : \delta$$



ACTION SCHEMA

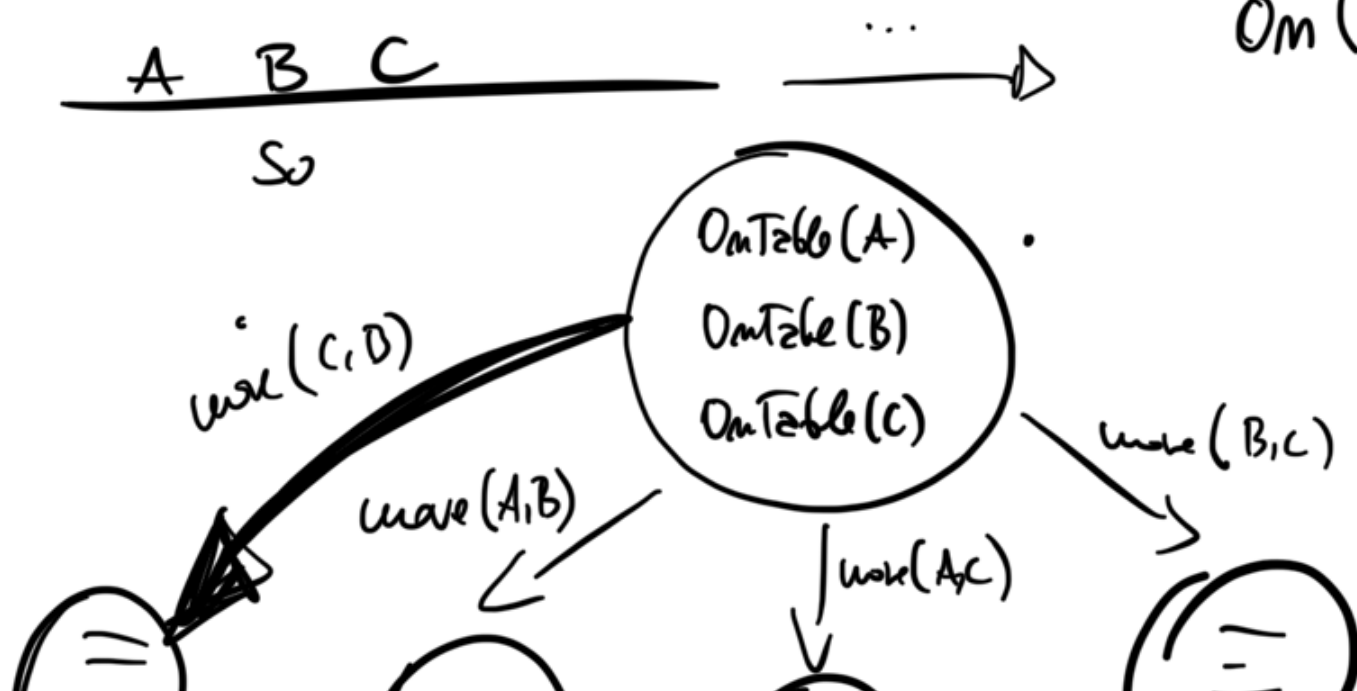
- name
- parameters
- precondition: bot. of true/false facts
- effects: ADD, DEL list

move(x,y)



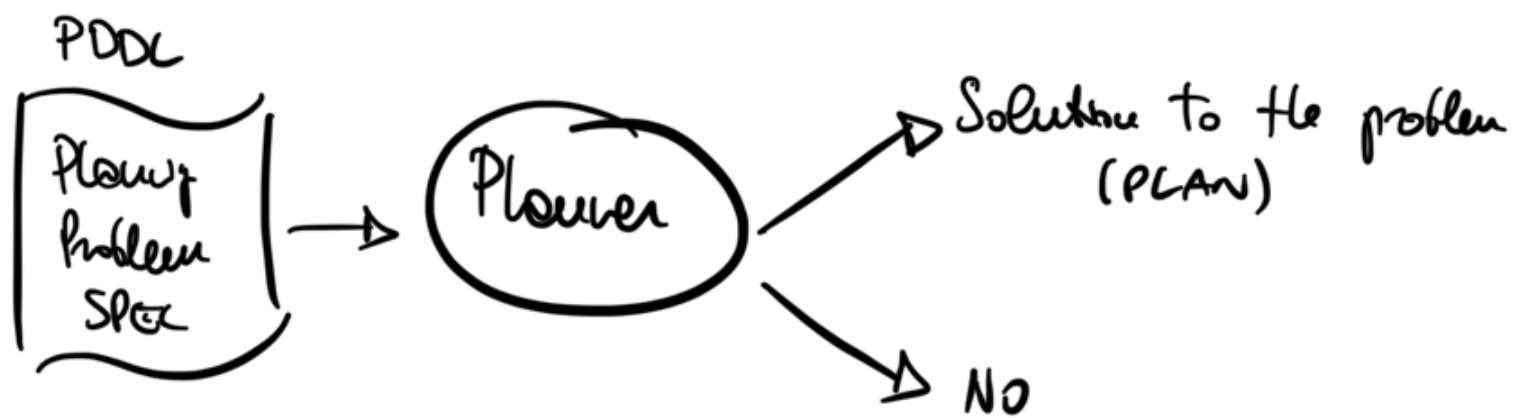
G

On(A,B)





PDDL : Planning Domain Definition Language



PDDL PLANNING PROBLEM: DOMAIN SPEC + PROBLEM SPEC

↑
essentially, action
schemas
(+ additional
stuff)

↑
- Objects (with type)
- initial state
- goal

DOMAIN SPECIFICATION

(define (domain blocksworld)

(:requirements :strips :typing :equality)

(:types block)

(:predicates

(on ?x ?y - block) ; x is on top of block y

(ontable ?x - block) ; x is on table

comment until end of line

(clear ?x - block) ; x is clear (no object on top)

(handempty) ; gripper is not holding anything

(holding ?x - block) ; gripper is holding block x
)

(:action pickup ; action to pickup a block from table

:parameters (?x - block)

:precondition (and
 (clear ?x)
 (ontable ?x)
 (handempty))
] pickup(A)

:effect (and
 (not (ontable ?x))
 (not (clear ?x))
 (not (handempty))
 (holding ?x))
] DEL:
(not (ontable A))
(not (clear A))
(not (handempty))
ADD:
(holding A)

(:action put-down ; puts block ?x on table

:parameters (?x - block)

:precondition (and (holding ?x))

:effect (and

(not (holding ?x))
 (handempty)
 (ontable ?x)
 (clear ?x))
)

(:action stack ; stack block ?x on top of ?y
 :parameters (?x ?y - block)
 :precondition (and (holding ?x) (clear ?y) (not (= ?x ?y)))
 :effect (and (handempty) (not clear ?y) (clear ?x)
 (on ?x ?y) (not (holding ?x))))

)
 (:action unstack ; unstacks block ?x from block ?y
 :parameters (?x ?y - block)
 :precondition (and (on ?x ?y) (clear ?x)
 (handempty))] π
 :effect (and (not (on ?x ?y)) (clear ?y)
 (not (handempty)) (holding ?x)
 (not (clear ?x)))] eff
)

PROBLEM SPECIFICATION

(define (problem blocks-world-problem)

(:domain blocksworld)

(:objects a b c - block)

(:init (handempty) (ontable a) (ontable b) (ontable c)
(clear a) (clear b) (clear c))

(:goal (and (clear c) (ontable b) (on c a) (on a b)))
)

Three FUNDAMENTAL PROBLEMS in Reasoning About Actions:

1. QUALIFICATION PROBLEM:

How to specify the conditions under which an action is executable.

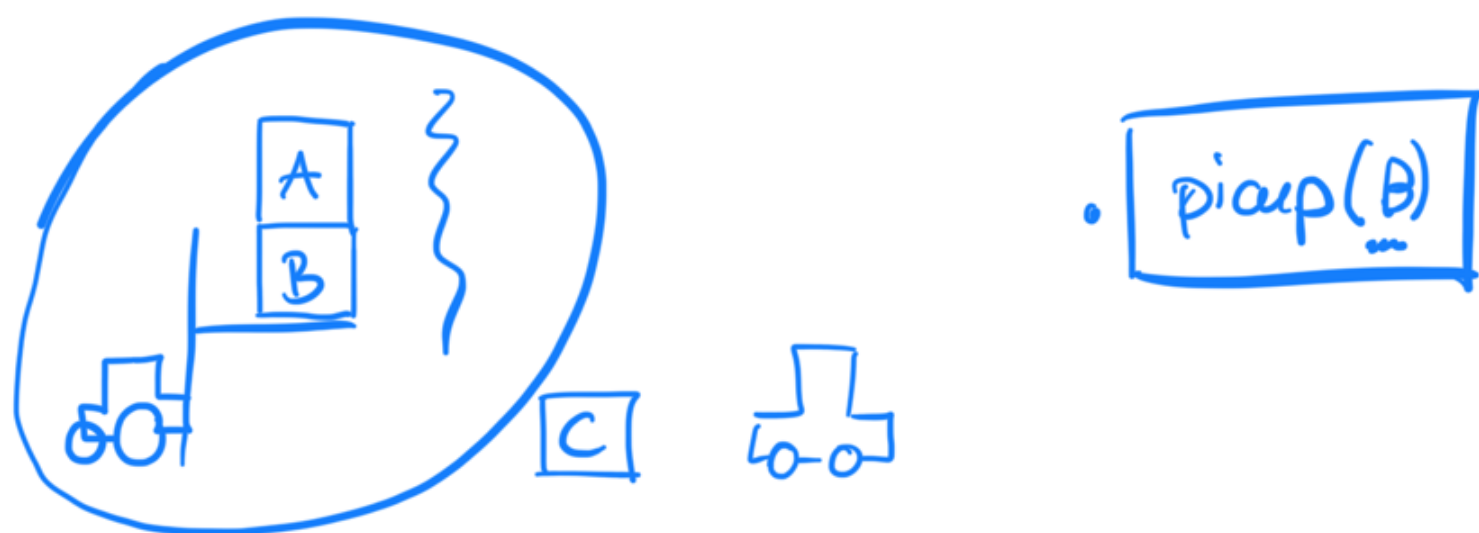
2. RAMIFICATION PROBLEM:

How to specify indirect effects.

3. FRAME PROBLEM

How to specify what happens after ...

action is executed. In particular, slot remains unchanged.



A

B

pickup(A) •

DIRECT: (not (handempty)) (holding A)
(not (clear A))

INDIRECT: (not (on A B)) (clear B) •

: EXICUS

PDDL
DOMAIN
PROBLEMS

(ontable A)
(ontable B)

pickup(A)

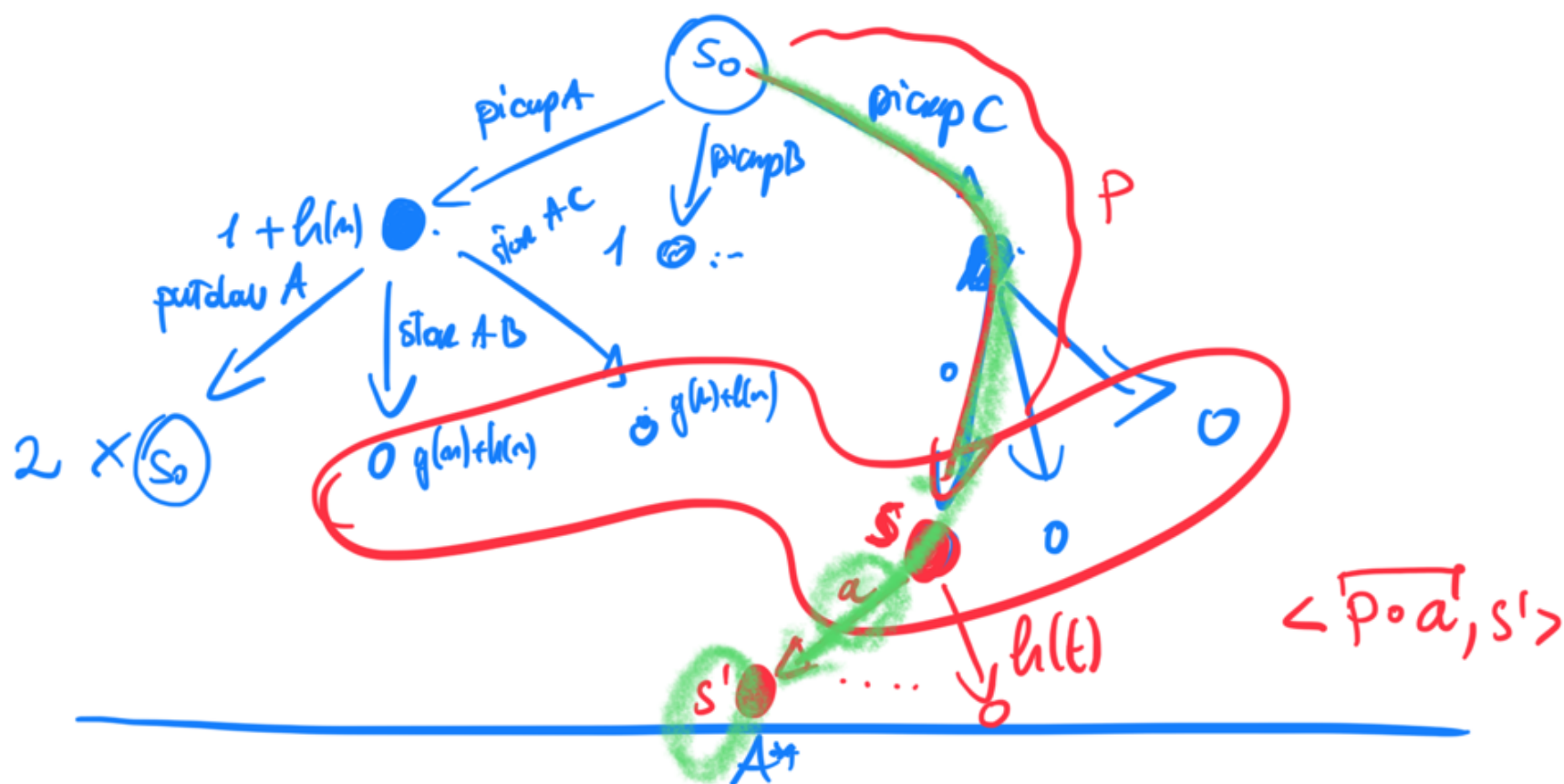
(ontable B)
ontable C)

store A B (—)

(low entropy)

(holding A)

store AC $\rightarrow$ $(\equiv)$

$$S' = (S \setminus DE) \cup AOD$$


$$\alpha(s) = \{a \mid a \in A \text{ and } a \text{ is applicable in } s\}$$

PDDL

Plom $A^*(\overline{D = \langle S, A, \alpha, \delta \rangle}, s_0, G \subseteq S, c, h) \{$

Marked = $\{ \langle \mathcal{E}, s_0 \rangle \}; \quad \mathcal{E}$: empty plan

$$\text{Fromber} = \{ \langle \mathcal{E}, s_0 \rangle \}_j$$

```

while (Frontier  $\neq \emptyset$ ) {

```

$$\text{extract} \langle p, s \rangle = \text{argmin}_{\langle q, t \rangle \in \text{Frontier}} \{c(q) + h(t)\}$$

```

if (s ∈ G) return D;

```

ρ

forall ($a \in \alpha(s)$) {

$s' = \delta(a, s);$

if (there is no p' s.t. $\langle p', s' \rangle \in \text{Marked}$) {

Add $\langle p \cdot a, s' \rangle$ to Marked;

Add $\langle p \cdot a, s' \rangle$ to Frontier;

}

else { // there exists $\langle p', s' \rangle \in \text{Marked}$

let $\langle p', s' \rangle \in \text{Marked};$

if ($c(p \cdot a) < c(p')$) {

replace $\langle p', s' \rangle$ with $\langle p \cdot a, s' \rangle$ in Marked;

if ($\langle p', s' \rangle \in \text{Frontier}$) {

remove $\langle p', s' \rangle$ from Frontier;

}

add $\langle p \cdot a, s' \rangle$ to Frontier;

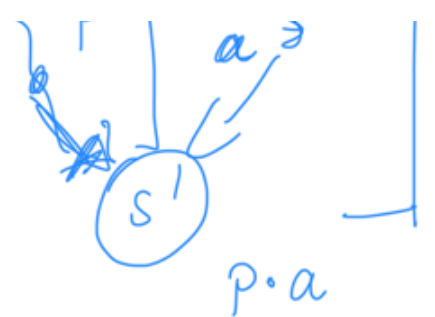
} // else

} // forall

} // while

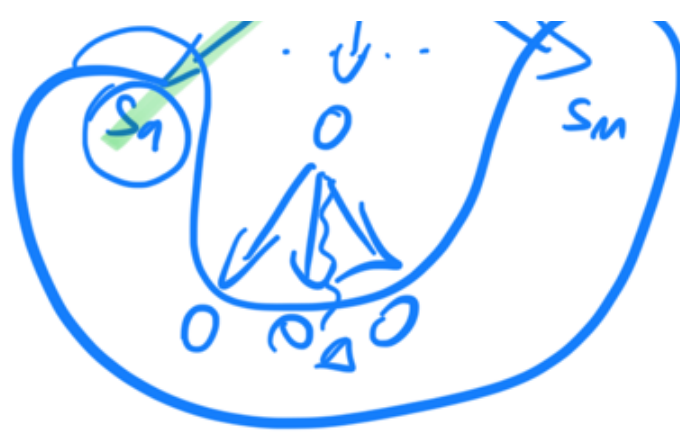
return failure;

}



$M = \{ \langle \epsilon, s_0 \rangle \}$

$F = \{ \langle a_1, s_1 \rangle, \dots \}$



$\langle Q_m, S_m \rangle$

PROBLEM-SPECIFIC HEURISTIC

3	7	1
2	4	5
8	6	

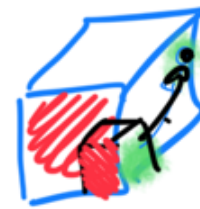


1	2	3
4	5	6
7	8	

e.g. # of misplacements wrt right position

• PROBLEM-INDEPENDENT HEURISTICS

- ignore preconditions
- ignore delete-lists



move (C_1, C_2)

- ignore delete-lists

Q_1 : pre: —
effect: $p(A), p(B)$

Q_2 : pre: —
effect: ~~not $p(A), p(C)$~~

Q_3 : pre: —

$p(A)$
 $p(B)$
 $p(C)$

Effect: $p(c)$

$(Q_1 Q_2)$

PDDL

STEPS: - positive preconditions

$(\text{not } (\text{on } A B) \dots)$

notOn : additional predicate which is the opposite of on

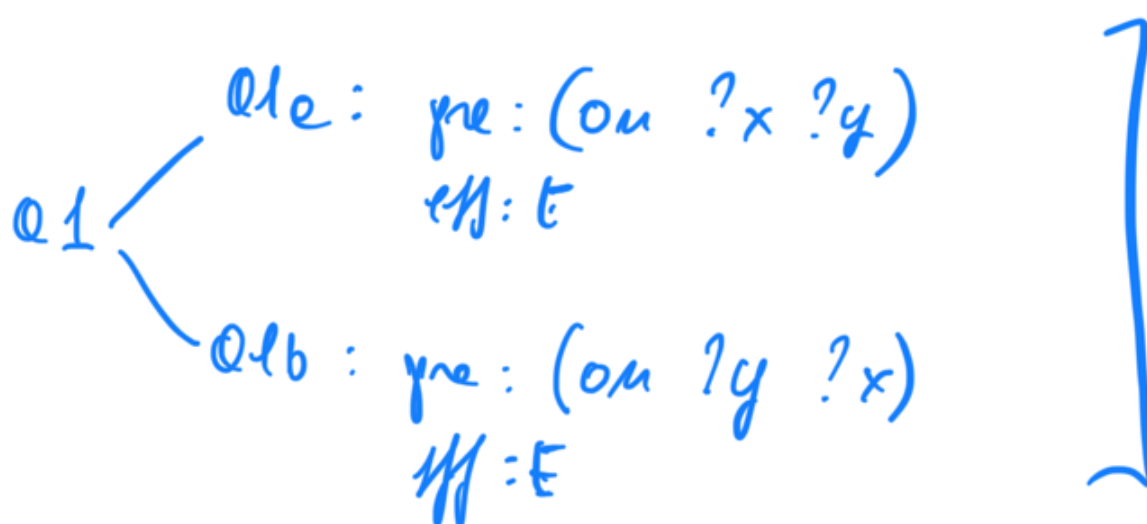
- no disjunctive preconditions

$Q1$:

$\text{pre}: (\text{or } (\text{on } ?x ?y) (\text{on } ?y ?x))$

(not allowed)

$\text{eff}: E$



- no quantifiers:

$\text{pickup}(A)$:

clear A

$\text{pre}:$

$\forall ?x \text{-block } (\text{not } (\text{on } ?x A)) \cdot$

Δ, B, C

$(\text{and } (\text{not } (\text{on } ?x A)) (\text{not } (\text{on } ?x B)) (\text{not } (\text{on } ?x C)) \dots)$

: typing : equality
 : negative-preconditions NEGATION in PRECONDITIONS
 : disjunctive-preconditions DISJUNCTION in PRECONDITIONS
 : quantified-preconditions

{ : existential-preconditions ENABLES $\exists$
 { : universal-preconditions ENABLES $\forall$

(: requirements : strips : typing ... : negative-preconditions...)

(: action stack

(: parameters ?x ?y-block)

(: precondition

(and (holding ?x)

$\phi(z, y)$

((forall ?z-block) (not (on ?z ?y)))

(not (exists ?z-block) (on ?z ?y)))

$\neg \phi(z, y)$

$\forall x \phi(x, y) \equiv \neg \exists x \neg \phi(x, y)$

... effects ...)

(: precondition

(or (on ?x ?y) (on ?y ?x))

(... ..)

CONDITIONAL EFFECTS

: conditional-effects

a:

if CONDITION holds in current state s

then EFFECT holds in next state s'



(: action pickup

(: parameters $?x$ - block)



(: precondition)

(: effect ~~(when (on B ?x) (holding B))~~

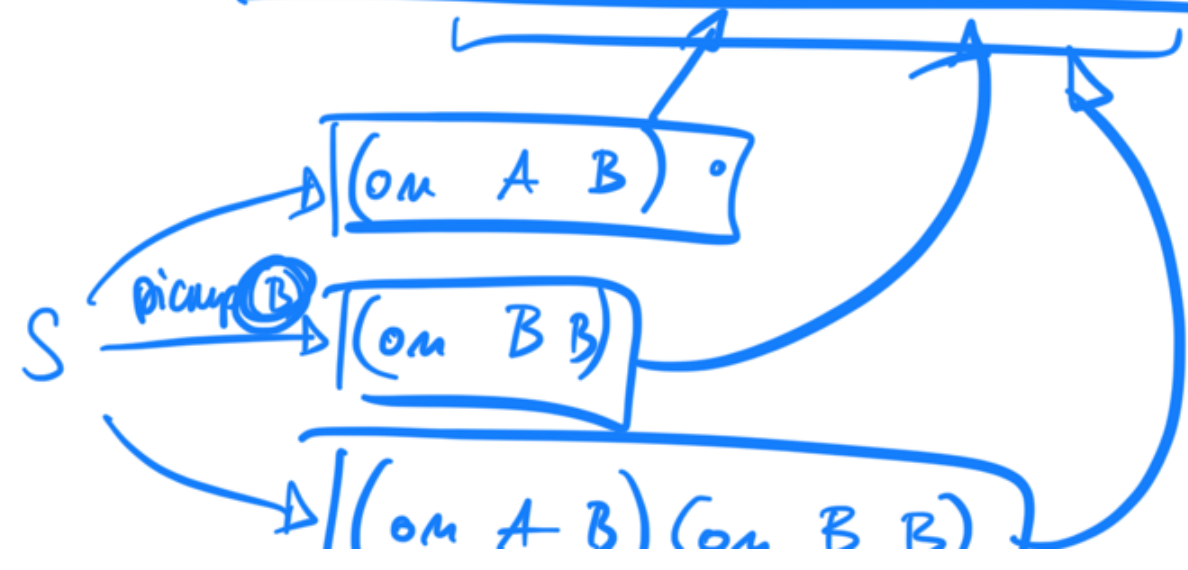
((forall ?b-blocks) (when (on ?b ?x) (holding ?b)))

(2nd (when (on A ?x) (holding A))

(when (on B ?x) (holding B))

:

(or (on A ^B?x) (on B ^B?x)))



Handwritten notes:

11

(or (one) (two) ... (six))

$(\lambda x. ((\text{exists } ?x) (\text{canExplode } (!x)))) (\text{boom})$

$S \longrightarrow S'$
base

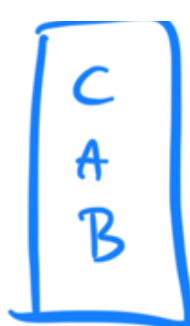
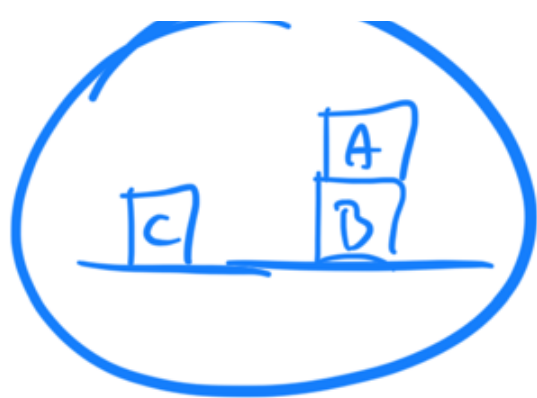
$S \longrightarrow S'$

:zde

ACTION DESCRIPTION LOGIC

(on A-B)

11



ONLINE RESOURCES

- planning.domains → ONLINE PLANNERS + PDDL RESOURCES
- planning.wiki → Planning documentation + PDDL docs
- www.fast-downward.org
- International Planning Competition (IPC)
 - Many pddl examples (Classical/Deterministic)

TRACE

FOND Planning

FULLY OBSERVABLE NON-DETERMINISTIC

